

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Arquitetura de Software
Parte 02

Arquitetura de Software

- O que é uma arquitetura de **microserviços**?
 - Segundo o **Martin Fowler**, ele cita que o termo de “**Arquitetura de microserviços**” descreve uma maneira particular de projetar aplicativos de software como **conjuntos de serviços implantáveis independentemente.**

Arquitetura de Software

- Imagine que você está construindo uma cidade.
 - Em vez de construir um enorme edifício que abriga todas as funções da cidade – como serviços de saúde, educação, transporte e comércio – você decide construir vários edifícios menores, cada um especializado em uma função específica.
 - Esta abordagem facilita a manutenção e a expansão da cidade, pois cada edifício pode ser atualizado ou substituído sem afetar os outros. Essa é a essência da arquitetura de microserviços.

Arquitetura de Software

- A **arquitetura de microserviços** é uma abordagem para o desenvolvimento de software onde uma aplicação é composta por **pequenos serviços independentes**, cada um executando uma função específica.
 - Estes serviços **comunicam-se entre si através de interfaces bem definidas**, geralmente APIs (Application Programming Interfaces).
 - Cada **microserviço é desenvolvido, implantado e escalado de forma independente**, o que permite uma maior flexibilidade e agilidade no desenvolvimento de software.

Arquitetura de Software

- Então uma arquitetura de microserviços adota serviços **fracamente acoplados**
 - que podem ser desenvolvidos, implantados e mantidos de forma independente.
- Cada um desses serviços é responsável por tarefas distintas e
 - pode se comunicar com outros serviços por meio de APIs simples para resolver um problema de negócios complexo maior.

Arquitetura de Software

- Características dos Microserviços
 - Os microserviços têm várias características que os distinguem de outras abordagens arquitetônicas:
 - 1. **Componentização por Serviço**: Cada componente (ou microserviço) é uma unidade funcional independente que realiza uma tarefa específica e bem definida.
 - 2. **Independência Tecnológica**: Cada microserviço pode ser desenvolvido com diferentes tecnologias, linguagens de programação e **bancos de dados**, dependendo das necessidades específicas da sua função.

Arquitetura de Software

- 3. **Desenvolvimento e Desdobramento Independentes:** Os microserviços podem ser desenvolvidos, testados, implantados e escalados de forma independente, sem a necessidade de coordenação estreita com outros serviços.
- 4. **Comunicação através de APIs:** A comunicação entre microserviços é feita através de APIs, geralmente usando protocolos HTTP/REST, gRPC ou **mensageria (como RabbitMQ ou Kafka).**

Arquitetura de Software

- Benefícios e Desafios Benefícios dos Microserviços
 - A adoção de uma arquitetura de microserviços traz vários benefícios significativos:
 - 1. **Escalabilidade**: Cada microserviço pode ser escalado de forma independente. Isso significa que você pode alocar mais recursos apenas para os serviços que precisam, otimizando o uso da infraestrutura.
 - 2. **Resiliência**: A falha de um microserviço não afeta os outros. Se um serviço falhar, os demais continuam a funcionar normalmente, aumentando a resiliência geral do sistema.

Arquitetura de Software

- 3. **Desenvolvimento Ágil**: As equipes podem trabalhar em diferentes microserviços de forma paralela e independente, o que acelera o ciclo de desenvolvimento e permite entregas contínuas.
- 4. **Facilidade de Manutenção**: Pequenas alterações podem ser feitas e implantadas em microserviços específicos sem a necessidade de recompilar e reimplantar toda a aplicação.
- 5. **Inovação Tecnológica**: A independência tecnológica permite experimentar e adotar novas tecnologias em serviços específicos sem impactar toda a aplicação.

Arquitetura de Software

- Desafios dos Microserviços
 - Apesar dos benefícios, a arquitetura de microserviços também apresenta desafios:
 - 1. **Complexidade Operacional**: Gerenciar e orquestrar vários serviços independentes pode ser complexo. É necessário um sistema robusto de monitoramento, logging e gerenciamento de falhas.
 - 2. **Comunicação entre Serviços**: A comunicação entre microserviços, especialmente em ambientes distribuídos, pode introduzir latência e complexidade adicional.

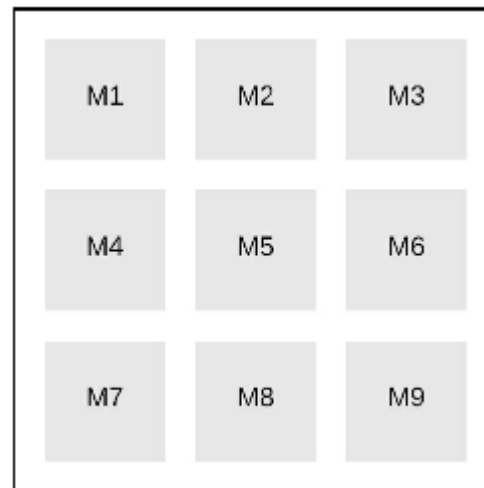
Arquitetura de Software

- 3. **Gerenciamento de Dados**: Cada microserviço pode ter seu próprio banco de dados, o que complica a manutenção da consistência e a integração dos dados entre serviços.
- 4. **Segurança**: Garantir a segurança de múltiplos pontos de entrada (APIs) pode ser mais desafiador do que em uma aplicação monolítica.

- Benefícios de uma arquitetura de microserviços
 - A separação da aplicação por **serviços independentes** permite **escalabilidade individual** de cada microserviço.
 - A **independência dos microserviços** permite que a aplicação **continue funcionando** caso ocorra erro em algum serviço,
 - ou seja, **apenas alguma funcionalidade** ficará indisponível ao invés de toda aplicação.
 - **Agilidade no deploy** de novas funcionalidades.
 - Permite **trabalhar com diferentes tecnologias** por microserviço.

Arquitetura de Software

- Arquitetura de Microserviços
 - Mesmo que um sistema tenha sido particionado em módulos, **estes executam como um monolito** (mesmo processo, mesma área de memória).



Monolito com nove módulos (componentes, pacotes, etc.) –

Exemplo – Aplicação web - MVC

Arquitetura de Software

- Problema
 - Uma mudança realizada em um módulo pode causar efeitos colaterais em outros módulos.
 - Por exemplo, módulos que compartilham uma variável global, ou um atributo estático.
 - Uma mudança nessa variável realizada em um módulo pode comprometer o funcionamento de outros.
 - Esse risco é maior do que se imagina.
 - Isso demanda um processo rigoroso e burocrático para o lançamento de novas releases (testes manuais de funcionalidades mais críticas).

Arquitetura de Software

- Desta forma, o desenvolvimento passou a ser ágil, mas a entrada em produção não acompanhou.

- Microserviços:

Módulos são executados em processos independentes, sem compartilhamento de memória.

- O sistema é decomposto em módulos não apenas no desenvolvimento, **mas também na execução.**
- Isto obriga a **redução do acoplamento** (dependência entre módulos.)

Arquitetura de Software

- Desta forma, **não há mais possibilidade de que um módulo acesse um recurso** interno de outro módulo, variável global ou atributo estático.
- Toda comunicação tem que ocorrer via **interfaces públicas dos módulos**.
- A obediência a essa regra é **garantida pelo sistema operacional**.

Arquitetura de Software

- O termo “micro” é usado porque os serviços não implementam funcionalidades complexas.
- Isto proporciona escalabilidade (crescer de acordo com o crescimento da demanda).
 - Os microsserviços permitem replicar apenas os componentes (serviços) diretamente relacionados com os problemas de performance (gargalos).

Arquitetura de Software

- É uma arquitetura que **possui serviços que são implantados de maneira independente** e que são modelados com base em um domínio de negócios.
- Os microsserviços expõem suas funcionalidades entre si e para sistemas web por meio de uma **arquitetura cliente-servidor (API REST - Representational State Transfer, em português Transferência de Estado Representacional)**
- Tal característica os torna um **sistema distribuído.**

Arquitetura de Software

- REST, ou Representational State Transfer, é um padrão entre sistemas web, facilitando a comunicação entre eles.
 - Os sistemas em conformidade com REST, muitas vezes conhecidos como sistemas RESTful
 - REST API, comunicação stateless (sem estado).
 - Requisição deve incluir todos os atributos necessários para a execução.

Arquitetura de Software

- REST indica então um conjunto de restrições que devem ser seguidas no desenvolvimento de uma aplicação na internet.
- Estas regras permitem o desenvolvimento de uma aplicação com interface bem definida,
 - rotinas padronizadas e facilmente representadas, que facilitam a comunicação entre máquinas e usuários

Arquitetura de Software

- No estilo REST, a implementação do **cliente e do servidor pode ser feita de forma independente**, sem que cada um conheça o outro.
 - Isso significa que o **código do lado do cliente pode ser alterado** a qualquer momento, sem afetar a operação do servidor, e o contrário também é válido.

Arquitetura de Software

- Além disso, **cada microserviço possui seu próprio banco de dados**, isto é, suas próprias tabelas.
- Se o ecossistema adotar o **compartilhamento de banco de dados**, o **acoplamento entre os sistemas se tornará alto**.

Arquitetura de Software

- Design orientado a domínios (DDD)
 - Para um ecossistema ser considerado um ecossistema de microserviço é preciso que a modelagem de cada sistema seja baseada em um domínio, isto é, é preciso que cada sistema represente um dos domínios do negócio.
 - Exemplo: a tabela cliente e as suas tabelas auxiliares (tabelas fortemente acopladas a ela) são um domínio de negócio, pois os desenvolvedores e usuários irão manipulá-las ao realizarem buscas, inserções, atualizações e exclusões por meio de um sistema.

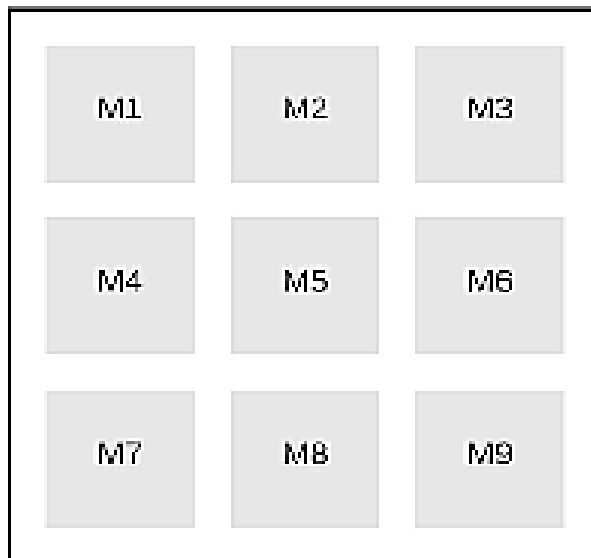
Arquitetura de Software

- Para um **sistema ser considerado um microserviço** é preciso que ele **não compartilhe seu banco de dados,**
 - ou seja, é preciso que somente ele consiga manipular diretamente determinadas tabelas.

Arquitetura de Software

- Exemplo:
 - Dois servidores (máquinas reais ou virtuais) executando todo o monolito, **para escalar conforme demanda**

Servidor 1



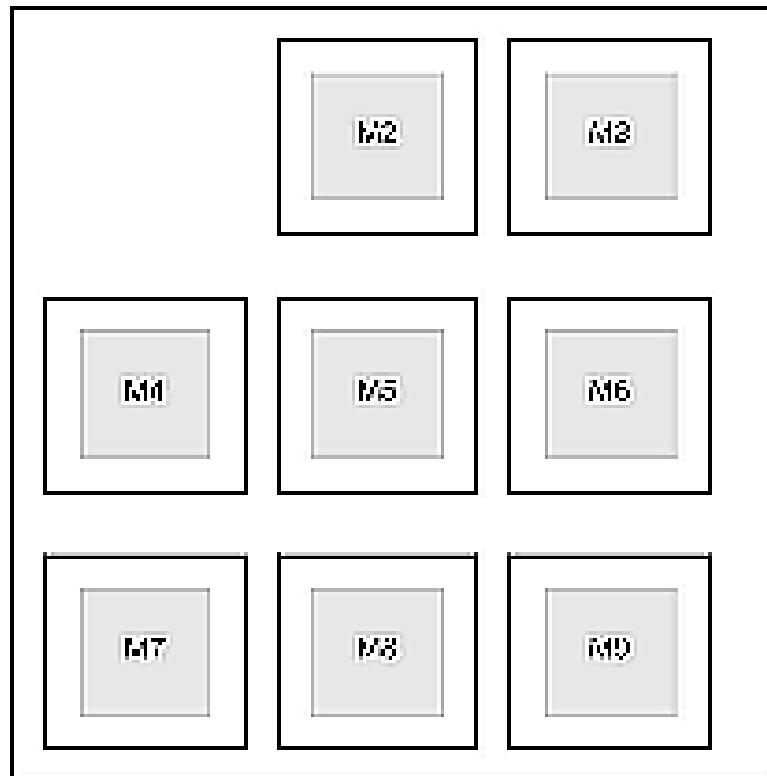
Servidor 2



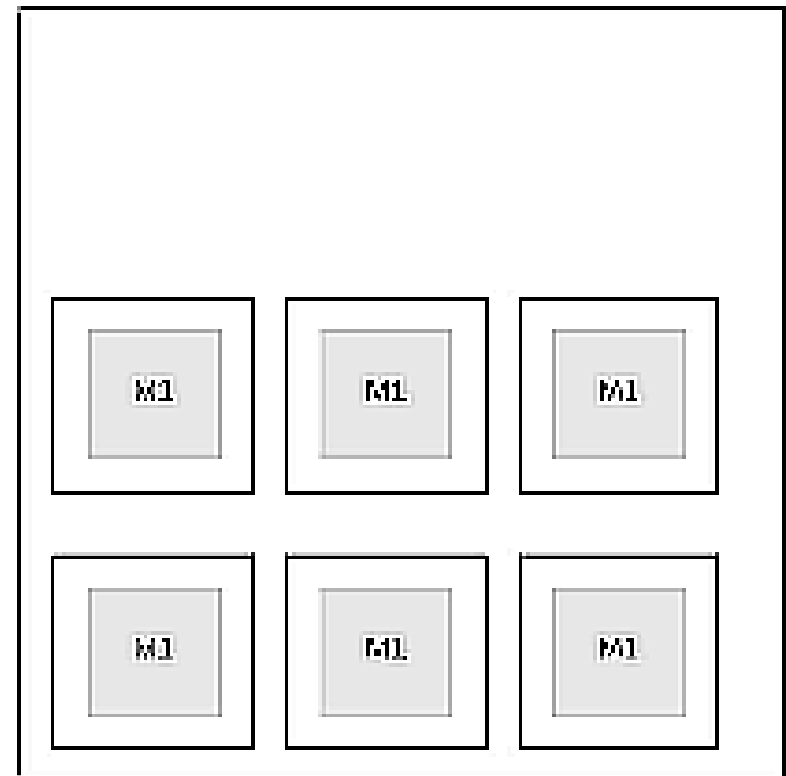
Arquitetura de Software

- Servidor 2 executando apenas microserviço M1, responsável por suposto gargalo no sistema

Servidor 1



Servidor 2



Arquitetura de Software

- Desta forma, arquitetura de microsserviços,
 - permite que times trabalhem de forma mais independente, em microsserviços específicos
 - permite escalar um sistema em um nível de granularidade mais fino do que é possível com monolitos.
 - os microsserviços podem ser implementados em tecnologias diferentes (sistemas heterogeneos)
 - em sistemas monolitos, falhas são totais.
 - em arquitetura de microsserviços, podemos ter falhas parciais.

Arquitetura de Software

- **Arquiteturas baseadas em microsserviços** tornaram-se possíveis e popularizadas devido às **plataformas de computação em nuvem**.
- Desta forma, **máquinas virtuais em plataformas de computação em nuvem** podem ser facilmente criadas a medida que a necessidade surge.
- Com isso fica **fácil escalar um microsserviço** utilizando-se novas máquinas virtuais.

Arquitetura de Software

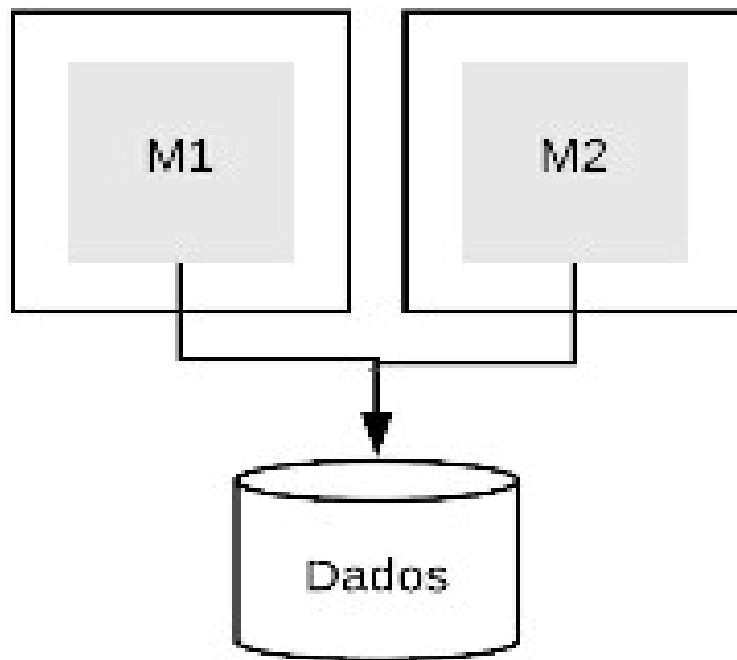
- Quanto mais VMs forem provisionadas, mais recursos serão consumidos.
 - Para cada **máquina virtual temos uma instalação completa do S.O.** que queremos virtualizar, além de ter o próprio hardware virtualizado.
 - Se é necessário por exemplo uma biblioteca comum para todas as máquinas virtuais, é preciso instalar em cada uma delas.
- **Modelo de contêiner** resolve esse problema
 - **Docker** é uma implementação desse modelo.

Arquitetura de Software

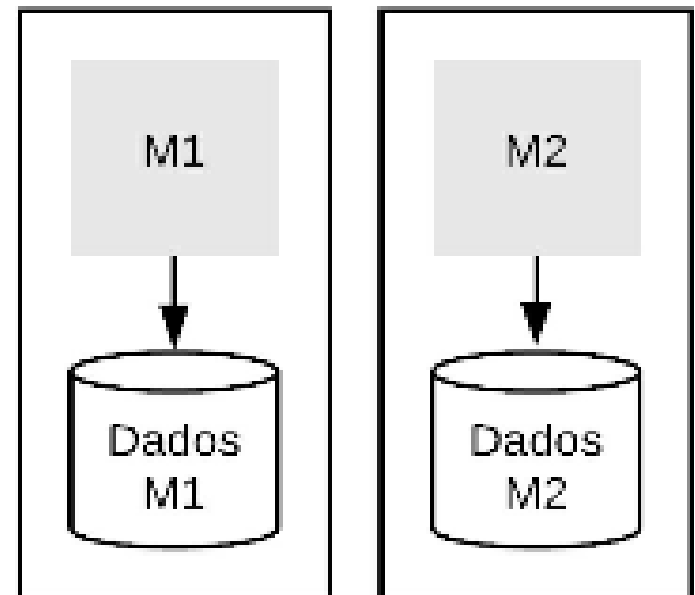
- **Microserviços devem ser autônomos** também do ponto de vista de dados.
- **Eles devem gerenciar os dados** de que precisam para prover o seu serviço.
- Desta forma, o seguinte cenário em que **dois microserviços compartilham o mesmo banco de dados** *não é recomendável*.

Arquitetura de Software

- Tal interdependência de dados é facilitada por modelos de dados leves, como bancos de dados NoSQL, JSON para troca de dados, etc.



M1 e M2 dependentes dos dados



M1 e M2 independentes dos dados

Arquitetura de Software

- **Frameworks** Java de para desenvolvimento de microsserviços
 - Spring Boot with Spring Cloud
 - Quarkus
 - Micronaut
 - Eclipse Vert.X
 - Oracle Helidon
 - AxonIQ
- *O Spring Boot é o mais popular Framework de para desenvolvimento de microsserviços.*

Arquitetura de Software

- Ferramentas para Implementação
- Docker é uma plataforma de contêineres que permite que você empacote aplicações e suas dependências em contêineres.
- Cada contêiner é uma unidade de software leve, portátil e auto-suficiente que pode ser executada em qualquer ambiente.
- Docker facilita a criação, implantação e execução de aplicações distribuídas, tornando-o uma ferramenta ideal para arquiteturas de microserviços.

Arquitetura de Software

- Benefícios do Docker:
 - **Isolamento**: Cada microserviço é executado em seu próprio contêiner, garantindo isolamento e evitando conflitos de dependências.
 - **Portabilidade**: Contêineres Docker podem ser executados em qualquer lugar, desde o laptop do desenvolvedor até servidores de produção na nuvem.
 - **Eficiência**: Contêineres compartilham o mesmo kernel do sistema operacional, o que os torna mais leves e eficientes em comparação com máquinas virtuais.

Arquitetura de Software

- Kubernetes
- Kubernetes, originalmente desenvolvido pela Google, é uma plataforma de orquestração de contêineres que automatiza a implantação, escalabilidade e gerenciamento de aplicações containerizadas.
- Kubernetes é amplamente utilizado para gerenciar aplicações de microserviços em ambientes de produção.

Arquitetura de Software

- Benefícios do Kubernetes:
 - **Orquestração**: Kubernetes automatiza a implantação e o gerenciamento de contêineres, garantindo que eles sejam executados de forma confiável e eficiente.
 - **Escalabilidade**: Kubernetes pode escalar automaticamente as aplicações com base na carga de trabalho.
 - **Auto-recuperação**: Kubernetes monitora o estado dos contêineres e reinicia aqueles que falham, garantindo alta disponibilidade.

Arquitetura de Software

- Exemplos Práticos Estudo de Caso:
- Netflix
 - Netflix é um dos exemplos mais emblemáticos de uma empresa que adotou a arquitetura de microserviços em grande escala.
 - Inicialmente, a Netflix operava com uma arquitetura monolítica, mas à medida que sua base de usuários crescia, surgiram desafios de escalabilidade e resiliência.

Arquitetura de Software

- A transição para uma arquitetura de microserviços permitiu que a Netflix escalasse rapidamente e mantivesse uma alta disponibilidade, mesmo com um aumento exponencial de usuários e tráfego.

Arquitetura de Software

- Implementação de Microserviços:
 - Cada função da Netflix, como recomendações de filmes, streaming de vídeo e gerenciamento de contas, é implementada como um microserviço separado.
- Ferramentas Utilizadas:
 - Netflix utiliza ferramentas como Docker para contêinerização e Kubernetes para orquestração, além de várias outras ferramentas internas e de código aberto para gerenciamento de microserviços.

Arquitetura de Software

- Estudo de Caso: Amazon
 - Amazon também adotou a arquitetura de microserviços para lidar com sua vasta gama de serviços e produtos.
 - Inicialmente, a Amazon operava com um monolito, mas à medida que expandia suas operações, a complexidade e a necessidade de escalabilidade levaram à adoção de microserviços.

Arquitetura de Software

- **Implementação de Microserviços**: Serviços como processamento de pagamentos, gerenciamento de inventário e recomendações de produtos são implementados como microserviços independentes.
- **Ferramentas Utilizadas**: Amazon utiliza sua própria plataforma de nuvem, AWS, juntamente com ferramentas como Docker e Kubernetes, para gerenciar sua arquitetura de microserviços.

Arquitetura de Software

- A **arquitetura de microserviços** representa uma mudança paradigmática na maneira como desenvolvemos e gerenciamos aplicações de software.
- Ao **dividir uma aplicação monolítica** em pequenos serviços independentes, podemos alcançar uma maior flexibilidade, escalabilidade e resiliência.
- No entanto, essa abordagem também **introduz novos desafios, como a complexidade operacional** e a necessidade de ferramentas robustas para orquestração e monitoramento.

Arquitetura de Software

- Ferramentas como **Docker e Kubernetes**
 - são **fundamentais** para a implementação eficaz de microserviços,
 - oferecendo soluções para **contêinerização** e
 - **orquestração** que simplificam o desenvolvimento, a implantação e o gerenciamento de aplicações distribuídas.
- Ao entender os benefícios e desafios da arquitetura de microsserviços e ao dominar as ferramentas disponíveis, estamos melhor equipados para aproveitar ao máximo essa abordagem moderna e poderosa para o desenvolvimento de software.

Arquitetura de Software

- **Comunicação entre microserviços:**
 - comunicação **síncrona** via HTTP (REST)
 - comunicação **assíncrona**
 - **via gRPC** (Google Remote Procedure Call) ou
 - **Message Brokers** (RabbitMQ, Apache Kafka, Apache ActiveMQ)

Exercícios

- A arquitetura de software é importante, pois ela é responsável pelo desempenho, a robustez e a capacidade de distribuição e manutenção de um sistema. Selecione a arquitetura que usa uma abordagem para criar um aplicativo para servidores como um conjunto de serviços pequenos.
- (A) Extreme Programming
- (B) Microserviços
- (C) Monolítica
- (D) TCP/IP
- (E) Ethernet

- RESPOSTA
- LETRA B

- A arquitetura de microsserviços:
- (A) é uma estrutura em que um aplicativo é separado em serviços menores e cada um desses serviços pode executar um processo exclusivo e, ainda, gerenciar seu próprio banco de dados.
- (B) é um estilo arquitetônico em que todos os componentes fazem parte de uma única unidade, sendo todos desenvolvidos e implantados em conjunto para que formem barramentos de serviços.
- (C) é uma estrutura que não necessita de clientes, pois os aplicativos se conectam à estrutura por meio de application programming interface (API) aos barramentos de serviços.
- (D) possui como vantagem a alta disponibilidade, pois o barramento de serviços une todos os componentes, tornando-os uma única unidade, contudo, os aplicativos executados nessa arquitetura são menos eficientes, no que se refere à velocidade, do que a monolítica.
- (E) é formada por componentes, acessados por meio de mensagens, que podem ser desenvolvidos separadamente, contudo, por serem stateless, ao serem reimplantados, ou se houver a necessidade de alteração de um serviço na arquitetura, é cogente reimplantar um aplicativo inteiro.

- RESPOSTA
- LETRA A

- Microserviços são uma abordagem arquitetônica que compõe o software em pequenos componentes ou serviços independentes, de forma que, no desenvolvimento e nas modificações do código, podem ser implantados serviços específicos de forma independente.
- C Certo
- E Errado

- RESPOSTA
- CERTO

- Assinale a alternativa que não corresponde a uma diretriz do desenvolvimento de software baseado na arquitetura de microsserviços.
- (A) Serviços devem ser implantáveis (deployable) de forma independente.
- (B) Serviços devem ser organizados ao redor de capacidades de negócio.
- (C) Deve-se utilizar um banco de dados relacional único para todos os serviços que compõem a aplicação.
- (D) Deve haver baixo acoplamento entre serviços.
- (E) Pode-se utilizar linguagens de programação distintas para implementar serviços diferentes, ainda que componham a mesma aplicação.

- RESPOSTA
- LETRA C

- Assinale a opção que representa uma desvantagem da arquitetura de microsserviços em relação a arquitetura monolítica.
- (A) Modularidade e reuso.
- (B) Flexibilidade tecnológica.
- (C) Escalabilidade.
- (D) Resiliência e tolerância a falhas.
- (E) Comunicação e latência.

- RESPOSTA
- LETRA E

- Sistemas de grandes empresas, como a Amazon ou o Facebook, dificilmente ficam fora do ar em sua totalidade. Normalmente, as falhas ficam restritas a partes desses sistemas. Por exemplo, o chat do Facebook pode sair do ar, mas isso não compromete o funcionamento de outros módulos da rede social, como o feed de notícias. Isso se deve principalmente à arquitetura adotada na implementação desses sistemas.
- Assinale o modelo arquitetural responsável por trazer essa robustez aos sistemas.
- A Microsserviços
- B Model-View Controller
- C Arquitetura em Camadas
- D Publish/Subscribe

- RESPOSTA
- LETRA A

- Acerca da arquitetura de microsserviços, assinale a opção correta.
- (A) A arquitetura de microsserviços é um padrão para a criação de aplicações distribuídas, porém não possui alta escalabilidade.
- (B) A comunicação entre os microsserviços é feita por meio de mecanismos padrões de tecnologia, como, por exemplo, o REST (representational state transfer).
- (C) Microsserviços utilizam uma única base de dados lógica para a persistência de dados.
- (D) Um requisito fundamental da arquitetura de microsserviços é o uso de versionamento de mudanças.
- (E) Os microsserviços são componentes autônomos e de alto acoplamento, de modo que há a necessidade de se utilizar uma mesma linguagem na sua construção.

- RESPOSTA
- LETRA B

- Microserviços são uma abordagem arquitetônica que compõe o software em pequenos componentes ou serviços independentes, de forma que, no desenvolvimento e nas modificações do código, podem ser implantados serviços específicos de forma independente.
- (C) Certo
- (E) Errado

- RESPOSTA
- CERTO

- Os microserviços criam vários serviços, cada qual executando em seu próprio processo, mas que não podem ser implantados de forma independente.
- (C) Certo
- (E) Errado

- RESPOSTA
- ERRADO

- Os microsserviços são módulos implantados e monitorados de forma independente, os quais são combinados, em uma coleção desacoplada, por aplicativos baseados nesses microsserviços.
- (C) Certo
- (E) Errado

- RESPOSTA
- CERTO

- Considerando-se a estrutura de microsserviços, os serviços podem ser escritos em diferentes linguagens de programação, mas devem usar a mesma técnica de armazenamento de dados.

(C) Certo

(E) Errado

- RESPOSTA
- ERRADO

- Uma arquitetura de microsserviços possui uma série de vantagens em relação às arquiteturas monolíticas. **NÃO** é uma dessas vantagens a
- (A) evolução mais rápida e independente do sistema.
- (B) redução da complexidade no desenvolvimento.
- (C) escalabilidade em um nível de granularidade mais fino.
- (D) possibilidade de implementar os serviços em tecnologias diferentes.
- (E) possibilidade de apresentar falhas parciais.

- RESPOSTA
- LETRA B

Arquitetura de Software

- Dúvidas?